

Event-Driven Architecture

Phase 3 · Advanced · Estimated time: 2 hours

1. Concept From First Principles

This chapter formalizes an architectural style this program has been using informally since Phase 2 Chapter 8: building a system around **events** — immutable facts about something that happened — rather than around direct, synchronous calls between services. **Event-Driven Architecture (EDA)** is the umbrella term for designing a system this way, and this chapter covers two of its most powerful, and most commonly confused, specific patterns: Event Sourcing and CQRS.

Event Sourcing: storing the history, not just the current state

Every database this program has covered so far stores **current state**: a user's balance is a single row with a number in it, and every update overwrites that number. **Event Sourcing** inverts this: instead of storing current state, you store the full, ordered, immutable sequence of events that *led to* that state — "AccountOpened," "MoneyDeposited: \$500," "MoneyWithdrawn: \$200," and so on, in an append-only **event store**. The current balance isn't stored directly at all — it's computed on demand by replaying all events for that account from the beginning (or from the last saved **snapshot**, an optimization to avoid replaying the entire history every time).

Backend engineer analogy

This is the conceptual difference between a bank statement (a running list of every transaction, from which your current balance can always be recomputed) and a single balance field in a spreadsheet cell that just gets overwritten each time. If you've ever wished your Eloquent model had a built-in, trustworthy audit log of every single change made to a row and why, rather than an **updated_at** timestamp with no history — event sourcing gives you exactly that, as the primary way the system stores data, not a bolted-on afterthought.

The genuine benefits are substantial: a complete, immutable audit trail comes for free (invaluable for financial systems and compliance); you can answer "what did this account's balance look like last Tuesday" by replaying events up to that point; and you can fix bugs in derived logic retroactively by simply replaying the unchanged historical events through corrected computation logic. The genuine costs are equally real: replaying long event histories can be slow without careful snapshotting; the mental model is unfamiliar and takes real team investment to adopt correctly; and querying "current state" efficiently (rather than historical facts) requires the second pattern this chapter covers.

CQRS: splitting how you write from how you read

CQRS (Command Query Responsibility Segregation) is the architectural principle that reading data and writing data are different responsibilities with different needs, and should be handled by separate models rather than forced through one shared model. A **command** (an intent to change state, like "WithdrawMoney") goes through a write path optimized for correctness and validation. A **query** (a request to read data, like "show me this account's current balance") goes through a completely separate read path, often backed by a different, denormalized, read-optimized data store — directly

echoing Phase 2 Chapter 14's denormalization argument, but elevated to a full architectural principle rather than a per-field decision.

CQRS and Event Sourcing are frequently used together, and this combination is where the real power (and real complexity) shows up: writes append immutable events to the event store (Event Sourcing), while a background process consumes those events and continuously updates one or more separate, denormalized **read models** (CQRS's query side) optimized for exactly the queries the application actually needs — a current-balance lookup table, a transaction-history view, a monthly-summary view, each independently derived from the same underlying event stream. This is precisely the async, eventually-consistent sync pipeline pattern from Phase 2 Chapters 13 and 14, and Chapter 6 of this Phase, applied at the level of a service's entire data model rather than just a search index or a single denormalized table.

It's important to be honest about when this combination is and isn't worth it, since "Event Sourcing + CQRS" is a genuinely trendy pair of terms that gets over-applied. The combination adds real, ongoing complexity: eventual consistency between the write side and every read model (a read model can briefly lag behind the latest event), more moving infrastructure pieces to operate and monitor, and a real learning curve for a team new to the pattern. It earns its cost specifically for domains where the audit trail, historical replay, and multiple independently-optimized read views are genuinely valuable business requirements — financial ledgers, order-management systems with complex state histories, inventory systems needing point-in-time reconstruction — not as a default architectural choice for every service, where a simpler CRUD-plus-cache model (Phase 1 Chapter 8) remains entirely appropriate.

2. Real-World Example / Case Study

Razorpay's and PhonePe's core payment ledger is a natural, textbook fit for Event Sourcing: every transaction is fundamentally an immutable fact ("-\$500 charged at 3:04pm"), a complete audit trail is a hard regulatory requirement, and the ability to reconstruct an account's exact historical state at any point for dispute resolution or compliance audits is genuinely valuable, not merely a nice-to-have.

A Flipkart- or Noon-scale order-management system, where an order moves through many states (placed, paid, packed, shipped, delivered, possibly returned or refunded) over days, benefits from CQRS's read/write split: the write side carefully validates and records each state transition as an event, while separate read models serve the customer-facing "track my order" view, the seller's "orders to fulfill" dashboard, and internal analytics, each independently optimized without forcing one shared schema to serve all three very different access patterns.

A simpler internal admin tool at a Careem- or Talabat-scale company — a config-flag management panel used by a handful of engineers — is a good example of where Event Sourcing/CQRS would be over-engineering: no meaningful audit-trail requirement, no multiple divergent read patterns, and a simple CRUD-plus-cache approach would serve the same need with far less operational overhead.

3. Diagram

Event Sourcing + CQRS, combined:

```
Command: "WithdrawMoney($200, account=42)"
|
v
[ Write Model / Command Handler ]
|   validates, then appends
v
[ Event Store (append-only, immutable) ]
  MoneyDeposited($500)
  MoneyWithdrawn($200)
  ...
  |
  | (async, event-driven sync -- Phase 2, Ch 8/13/14 pattern)
  v
[ Read Model 1: current balances table ] <- fast lookups
[ Read Model 2: transaction history view ] <- for statements
[ Read Model 3: monthly summary view ]   <- for reporting

Query: "What is account 42's balance?" -> hits Read Model 1 directly,
NEVER replays the full event history for a simple lookup.
```

4. Trade-offs Table

Pattern	Benefit	Cost
Event Sourcing	Complete audit trail, point-in-time reconstruction, retroactive bug fixes	Replay cost, unfamiliar mental model, needs snapshotting
CQRS alone	Independently optimized read/write models	Two models to maintain instead of one
Event Sourcing + CQRS together	Full history plus multiple fast, purpose-built read views	Eventual consistency between write and read sides; real operational complexity
Plain CRUD (no ES/CQRS)	Simple, familiar, low overhead	No audit trail, no historical replay, one model must serve all reads

5. Common Interview Questions

Q1: "Explain Event Sourcing and how it differs from a normal database update."

Model approach: Contrast storing current state (overwritten on each update) with storing an immutable sequence of events, computing current state by replaying events, and name the audit-trail and point-in-time-reconstruction benefits as the concrete payoff.

Q2: "What is CQRS, and why would you split reads and writes into separate models?"

Model approach: Explain that reads and writes have genuinely different needs (validation-heavy vs. query-shape-optimized), and that forcing both through one shared model is a compromise that hurts both sides, whereas separate models let each be optimized independently, at the cost of eventual consistency between them.

Q3: "When would you NOT use Event Sourcing + CQRS, even though it sounds powerful?"

Model approach: Name a concrete case — a simple internal tool with no audit-trail requirement and a single, simple read pattern — and argue plainly that the added eventual-consistency and operational complexity isn't justified there, versus a plain CRUD-plus-cache approach.

Q4: "How do you avoid replaying an account's entire event history every time you need its current balance?"

Model approach: Describe snapshotting (periodically saving computed state so replay only needs to start from the last snapshot forward) as the standard mitigation for the replay-cost problem, and separately mention that a CQRS read model can also just maintain the current balance directly, updated incrementally as new events arrive, avoiding full replay for routine reads entirely.

6. Practice Exercises

- 1 Design the event schema (list the specific event types) for an order-management system using Event Sourcing, from order placement through delivery or return.
- 2 Explain, step by step, how a CQRS read model for 'orders to fulfill by restaurant' would be built and kept in sync from an event stream of order-status-change events.
- 3 Argue both sides: should a simple user-settings service (toggle notification preferences) use Event Sourcing + CQRS? Give your final recommendation with reasoning.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Components of CQRS and Event Sourcing Architecture - How it Works"	Walks through the components of both patterns together.
English	"Event Sourcing 101: Command Query Responsibility Segregation (CQRS)"	Introduces both patterns and how they complement each other.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers event-driven architecture as part of a full bilingual course.
Hindi	"Complete System Design Roadmap 2026 HLD & LLD" — CodeHelp	Places Event Sourcing/CQRS in context of the broader Hindi HLD roadmap.

8. Key Takeaways

- Event-driven architecture builds systems around immutable events rather than direct, synchronous calls.
- Event Sourcing stores the full history of events, computing current state by replaying them, giving a built-in audit trail.
- CQRS separates the write model (commands) from the read model (queries), letting each be optimized independently.
- Event Sourcing + CQRS together let one event stream feed multiple independently-optimized read models asynchronously.
- Snapshotting avoids replaying an entire event history on every read.

- The combination adds real complexity (eventual consistency, more infrastructure) and isn't a default choice.
- Reserve it for domains where audit trails, historical replay, or multiple divergent read patterns are genuine requirements.